



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

责任链模式（Chain of Responsibility）

一、意图（Intent）

在嵌入式 C 里，责任链的核心不是“对象一个接一个”，而是：

把同一入口收到的请求交给一串处理节点依次试判和接管，应用层不直接决定该找哪个模块、哪个驱动、哪个协议分支。

它控制的是三件事：

- 谁有资格先看这个请求
- 谁有资格真正接管它
- 当当前节点不处理时，请求按什么顺序继续向后传递

所以从嵌入式角度看，责任链首先是在控制**请求分派权**，其次是在控制**状态判断顺序**。

二、动机（Motivation）

嵌入式系统里最常见的坏味道之一，就是把“谁来处理请求”直接写死在发送方里：

- `if/else` 决定交给哪个模块
- 按状态位判断哪个模块当前可用
- 按优先级手写多层回退逻辑
- 在多个文件里散落“如果 A 不行就试 B”

这种写法在 PC 软件里已经不好维护，在嵌入式里问题会更明显：

1. 分派规则和业务动作混在一起

应用层本来只该表达“我要处理这个事件”，结果却顺手承担了：

- 状态裁决
- 模块选择
- 降级路径
- 默认兜底

这样一来，应用层会知道太多驱动层和中间层细节。

2. 可裁剪性差

很多嵌入式项目需要按芯片型号、资源等级、SKU 做裁剪。

如果分派顺序写死在业务层，那么删掉一个模块、替换一个协议处理器、调整优先级，往往都要改上层代码。

3. 时序与资源判断容易散开

真实系统里，一个请求能否处理，通常不只取决于“功能上支持不支持”，还取决于：

- 模块是否初始化完成
- 当前是否 busy
- 总线是否占用
- 电源域是否已打开
- 当前上下文是不是 ISR

这些判断若散落在不同调用点，系统会越来越难排查。

4. 默认行为很难统一

很多开源项目和厂家 SDK 的默认做法，都是先提供**统一入口**，再让若干候选处理节点按顺序试判，而不是让发送方直接绑定某个具体实现。

责任链就是把这部分“谁接管请求”的职责，从发送方挪到链本身。

三、适用性（Applicability）

当系统满足下面这些条件时，责任链就很合适：

- **有统一入口**：请求都从 UART、USB、事件中心、协议层入口进入
- **有多级判断**：是否支持、是否就绪、是否允许、是否降级都要判断
- **有顺序要求**：必须先试主处理器，再试备用处理器，再试兜底处理器
- **需要可扩展**：后续可能新增一个处理节点，但不想改发送方
- **需要可裁剪**：不同产品线启用不同节点组合

嵌入式里常见的判断语句不是“这个模式像不像”，而是：

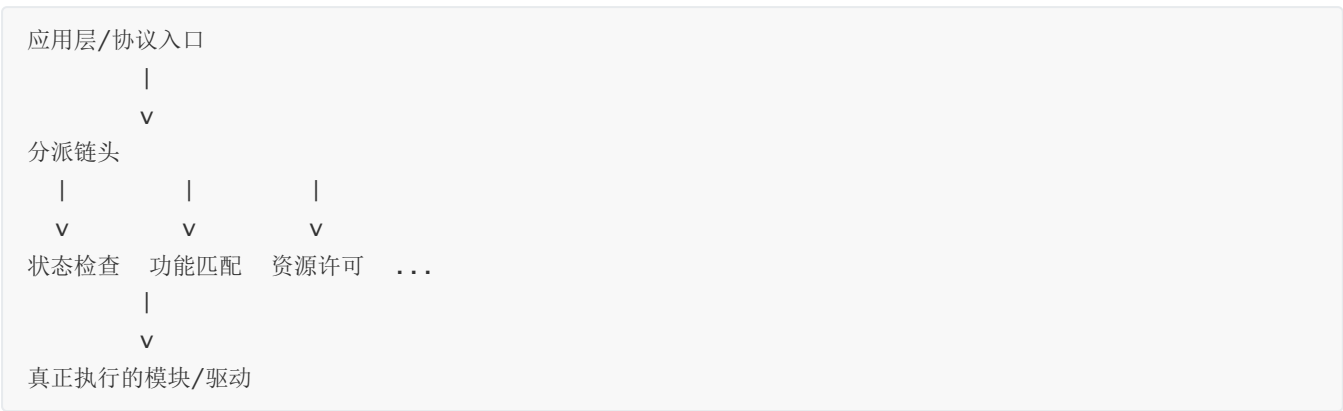
请求有多个候选接收者，且接收权要按顺序在运行时决定。

四、结构（Structure）

从嵌入式 C 的视角，可把它看成下面这条路径：



若把工程层次写清楚，通常会更像这样：



每个节点只做两件事：

1. 判断自己现在是否应该接管
2. 若不接管，就把请求交给下一个节点

在 C 里，链不一定非要真用链表。

常见实现有三类：

- 结构体 `next` 指针串链
- 静态处理表按顺序遍历
- 编译期注册段 + 运行期顺序遍历

五、参与者（Participants）

1. Handler（处理节点接口）

它定义统一的处理入口。

在嵌入式 C 里，通常体现为：

- 统一函数原型
- 统一节点结构体

- 统一返回值约定

例如概念上会有这样的语义：

- `HANDLED`：我已经接管
- `PASS`：我不处理，继续传递
- `ERROR`：发现错误，直接终止

这里最重要的不是“接口长什么样”，而是：

链上的所有节点都必须用同一种处理语义说话。

2. ConcreteHandler（具体处理节点）

具体节点负责自己的那一段判断逻辑，例如：

- 只处理某类命令字
- 只处理某类设备地址
- 只在模块 ready 时接管
- 只在资源未占用时接管

它不应该知道全局所有规则，只需要知道：

- 我的受理条件
- 我的执行动作
- 不处理时把请求交给谁

3. Request / Context（请求与环境）

嵌入式里很多链式处理都离不开上下文，因此通常会有：

- 请求本体
- 当前解析位置
- 调用上下文（任务态 / ISR）
- 权限或来源信息
- 临时状态标志

责任链是否稳定，往往取决于这份上下文是否设计清楚。

4. Client（请求发起方）

发起方只负责把请求送进链头。

它最好只知道“统一入口”，而不知道后面有几个节点、谁先谁后、谁会最终处理。

六、协作（Collaborations）

责任链在嵌入式系统中的执行路径一般是：

1. 应用层、协议层或中断后半部生成一个请求

2. 请求被送入链头
3. 当前节点检查自己是否满足接管条件
4. 若满足，则在当前节点完成动作并结束
5. 若不满足，则返回“继续传递”
6. 后继节点重复同样流程
7. 最终由某个节点处理，或由兜底节点给出默认结果

这里真正关键的是：

发送者只负责“提交请求”，接管决策沿链传递。

这比在业务层写一堆“先试 A，再试 B，再试 C”更适合长期维护。

七、效果（Consequences）

收益

1. 发送方与具体处理模块解耦

应用层不再知道到底是哪个驱动、哪个协议子模块、哪个回退分支最终接管。

2. 处理顺序可集中管理

优先级、降级顺序、兜底行为都收敛到链中，而不是散落在业务代码里。

3. 更适合裁剪和扩展

新增一个节点、删掉一个节点、交换两个节点顺序，通常不需要改请求发送方。

4. 更符合嵌入式的资源准入逻辑

很多处理并不是“功能上能不能做”，而是“当前状态下允不允许做”。

责任链很适合承载这种逐级准入判断。

代价

1. 跟踪路径会变长

链长了以后，要查“为什么最后是这个节点接管”会变复杂。

2. 返回语义必须统一

一旦不同节点对 `PASS` / `HANDLED` / `ERROR` 的理解不一致，链就会非常难调。

3. 可能引入额外遍历开销

在极端高频路径上，链太长会带来可见的时间成本。

八、实现要点 (Implementation)

1. 优先静态组织，少做动态拼链

大多数 MCU 项目更适合：

- 静态数组
- 编译期注册表
- 固定顺序链

这样更容易控制 RAM、启动顺序和可预测性。

2. 明确返回值协议

务必在接口层规定清楚：

- 什么叫“我已处理”
- 什么叫“继续传递”
- 什么叫“直接失败”
- 出错后是否允许后继继续尝试

3. 不要让节点偷改全局状态

链式处理最怕某个节点在返回 `PASS` 前已经改了一半系统状态，后续节点又继续处理，最后留下脏状态。

4. 注意 ISR 与任务上下文边界

若请求从中断进入链，链上节点最好只做轻量判定，把真正重动作延后到任务上下文。

5. 兜底节点要明确

链尾到底是：

- 返回未支持
- 返回参数错误
- 返回默认处理
- 打日志并丢弃

一定要在设计阶段讲清楚。

九、典型应用

下面这些都不是为了讲模式硬凑出来的例子，而是很多开源项目和厂家默认就采用的组织方式：

1. USB 设备栈的控制请求分派

很多开源 USB Device 栈和厂家 USB 中间件，都会把 EP0 的 Setup 请求先送入统一入口，再依次判断：

- 标准请求
- 类请求
- 厂商自定义请求
- 用户兜底处理

哪个节点先匹配成功，哪个节点就接管请求。
这本质上就是责任链。

2. 内核/底层的通知链机制

像 Linux kernel 的 notifier 机制，本质上也是事件进入统一入口后，沿已注册处理者顺序通知，某些场景下还能中止后续传播。

3. GUI / 输入事件冒泡

很多嵌入式图形框架和厂家 HMI 方案里，一个输入事件会先交给当前控件，再按父层级向上冒泡，直到某一级消费掉事件。

这也是责任链非常典型的形态。

4. 资源准入检查链

很多产品里，一次“执行命令”并不会立刻碰硬件，而是先经过：

- 参数合法性检查
- 当前模式检查
- 电源域检查
- 外设 busy 检查
- 最终驱动执行

这类多级准入非常适合责任链。

总结

责任链在嵌入式 C 里的本质，就是把“谁来接管请求”的决定权从发送方拿出来，交给一条按顺序试判的处理链。